Assignment 1 (Cloud Services & Security)

PRN No. - 260840127015 Name - Khendake Pradip Goraksh Course - PGCP-HPCSA

Git Assignment – I

1. **Create a Local Git repository**
2. **Add Hello.java**
3. **Commit Hello.java**
4. **Add File.txt & modify Hello.java**
5. **Commit both the files**
6. **Create a Github repository Push all files from Local repository to Github**

```
pradip@ProBook:~$ pwd
/home/pradip
pradip@ProBook:~$ cd
pradip@ProBook:~$ mkdir git_practical
pradip@ProBook:~$ cd git_practical
```

```
pradip@ProBook:~/git_practical$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/pradip/git_practical/.git/
```

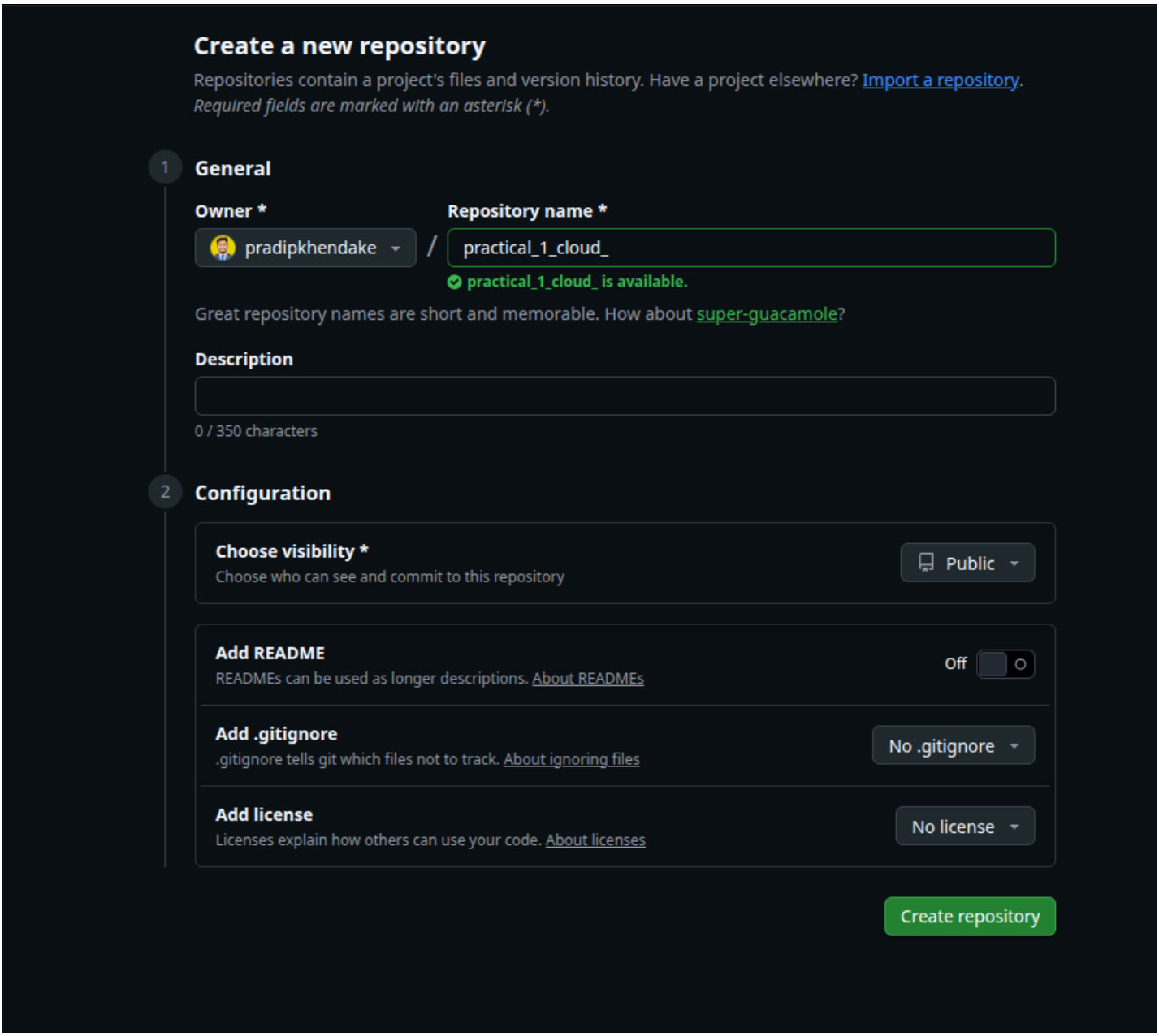
```
pradip@ProBook:~/git_practical$ nano Hello.java
```

```
public class Hello{
        public static void main (String[] args){
                System.out.println("Hello, World!");
        }
}
```

```
pradip@ProBook:~/git_practical$ git add Hello.java
pradip@ProBook:~/git_practical$ git commit -m "First Version"
[master (root-commit) 568c1e2] First Version
 1 file changed, 5 insertions(+)
 create mode 100644 Hello.java
```

```
pradip@ProBook:~/git_practical$ nano file1.txt
pradip@ProBook:~/git_practical$ nano Hello.java
pradip@ProBook:~/git_practical$ git add *
```

```
pradip@ProBook:~/git_practical$ git commit -m "Second Version"
[master 89fa646] Second Version
 2 files changed, 3 insertions(+)
 create mode 100644 file1.txt
```




```
pradip@ProBook:~/git_practical$ git remote add origin https://github.com/pradipkhendake/practical_1_cloud.git
pradip@ProBook:~/git_practical$ git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 633 bytes | 633.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/pradipkhendake/practical_1_cloud.git
 * [new branch]      master -> master
```

Git Assignment – II

Assignment 1: (Local Git)

1. Create a folder for local git repository.

2. Create a file called calculator.py with a simple function (add):

3. Stage and commit this file

4. In the same repository, create Greeting.java with accepts username and shows a personalised Hello message:

5. Stage and commit this file with a clear message.

6. Run git log --oneline to see their clean, two-step development history.

```
pradip@ProBook:~/java_docker_compose$ cd
pradip@ProBook:~$ mkdir local_git
pradip@ProBook:~$ cd local_git/
pradip@ProBook:~/local_git$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/pradip/local_git/.git/
pradip@ProBook:~/local_git$ nano calculator.py
pradip@ProBook:~/local_git$ git add *
pradip@ProBook:~/local_git$ git commit -m "added add calculator"
[master (root-commit) 15b19cd] added add calculator
 1 file changed, 9 insertions(+)
 create mode 100644 calculator.py
pradip@ProBook:~/local_git$ nano Greeting.java
pradip@ProBook:~/local_git$ git add *
pradip@ProBook:~/local_git$ git commit -m "clear"
[master 0a6376f] clear
 1 file changed, 14 insertions(+)
 create mode 100644 Greeting.java
pradip@ProBook:~/local_git$ git log --oneline
0a6376f (HEAD -> master) clear
15b19cd added add calculator
pradip@ProBook:~/local_git$
```

```python
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        return

add(a,b)
```

```java
import java.util.Scanner;

public class Greeting {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your username: ");
        String username = scanner.nextLine();

        System.out.println("Hello, " + username + "! Welcome to the repository.");

        scanner.close();
    }
}
```

Assignment 2: Branch Workflow

1. Rename the master branch to main using the following command
git branch -m master main
2. Start on the main branch. Create and switch to a new branch called
3. Open calculator.py and add a subtraction function:
4. Stage and commit this change only on the feature branch.
5. Switch back to main (git checkout main). Open calculator.py to observe that the subtraction function is gone .
6. Bring the new function into the main codebase by running git merge command.

```
pradip@ProBook:~/local_git$ git branch -m master main
pradip@ProBook:~/local_git$ git branch
* main
pradip@ProBook:~/local_git$ git checkout -b new_branch
Switched to a new branch 'new_branch'
pradip@ProBook:~/local_git$ git branch
  main
* new_branch
pradip@ProBook:~/local_git$ nano calculator.py
```

```
  GNU nano 7.2                          calculator.py *
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        print(a - b)
        return

add(a,b)
```

```
pradip@ProBook:~/local_git$ git add *
pradip@ProBook:~/local_git$ git commit -m "commit from feature branch."
[new_branch 2eac67b] commit from feature branch.
 1 file changed, 1 insertion(+)
pradip@ProBook:~/local_git$ git checkout main
Switched to branch 'main'
pradip@ProBook:~/local_git$ cat calculator.py
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        return

add(a,b)
```

```
pradip@ProBook:~/local_git$ git merge new_branch
Updating 0a6376f..2eac67b
Fast-forward
 calculator.py | 1 +
 1 file changed, 1 insertion(+)
pradip@ProBook:~/local_git$ cat calculator.py
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        print(a - b)
        return

add(a,b)
```

Assignment 3: Merge Conflict Resolution

1. On the main branch, ensure Greeting.java prints "Hello, yourname!".

2. Developer A (Branch 1): Create a branch. Edit the print line to say "Good morning, yourname.". Commit this change.

3. Developer B (Main Branch): Switch back to main. Edit the exact same line "Hey there, Friend!". Commit this change.

4. Try to merge the branch.

5. Resolve the conflict and merge the branch

```
pradip@ProBook:~/local_git$ nano *.java
pradip@ProBook:~/local_git$ git branch
* main
  new_branch
pradip@ProBook:~/local_git$ git checkout new_branch
M       Greeting.java
Switched to branch 'new_branch'
pradip@ProBook:~/local_git$ nano *.java
pradip@ProBook:~/local_git$ git add *
pradip@ProBook:~/local_git$ git commit -m "Commit from Developer 1"
[new_branch db3ecab] Commit from Developer 1
 1 file changed, 2 insertions(+), 1 deletion(-)
pradip@ProBook:~/local_git$ git checkout main
Switched to branch 'main'
pradip@ProBook:~/local_git$ nano *.java
pradip@ProBook:~/local_git$ git merge new_branch
Updating 2eac67b..db3ecab
error: Your local changes to the following files would be overwritten by merge:
        Greeting.java
Please commit your changes or stash them before you merge.
Aborting
pradip@ProBook:~/local_git$ git add *
pradip@ProBook:~/local_git$ git commit -m "Commit from Developer 2"
[main d358c39] Commit from Developer 2
 1 file changed, 1 insertion(+), 1 deletion(-)
pradip@ProBook:~/local_git$ git merge new_branch
Auto-merging Greeting.java
CONFLICT (content): Merge conflict in Greeting.java
Automatic merge failed; fix conflicts and then commit the result.
pradip@ProBook:~/local_git$ nano *.java
```

Assignment 4: Pull Request (GitHub)

1. Create a GitHub repository and link their local work

```
pradip@ProBook:~/local_git$ git remote add origin https://github.com/pradipkhendake/assignment_1_cloud-.git
pradip@ProBook:~/local_git$ git branch -M main
pradip@ProBook:~/local_git$ git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 12 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 1.32 KiB | 1.32 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/pradipkhendake/assignment_1_cloud-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Assignment 5: Collaboration Simulation: (A , B)

Before coding, B needs access to the project.

1. Add as Collaborator: A goes to the repository on GitHub → Settings → Collaborators → Click Add People and invite B.

2. Clone the Repo: B checks their email, accepts the invite, and clones the repository to their local machine:

```
pradip@ProBook:~$ git clone https://github.com/pradipkhendake/assignment_1_cloud-.git
Cloning into 'assignment_1_cloud-'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 12 (delta 1), reused 12 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (12/12), done.
Resolving deltas: 100% (1/1), done.
```

The Branch Workflow on Local Machine

B must never write code directly on the main branch.

1. Create a Branch: B creates a new branch for their specific task:

2. Write the Code: B edits the Java or Python files.

```
pradip@ProBook:~$ cd assignment_1_cloud-/
pradip@ProBook:~/assignment_1_cloud-$ ls
calculator.py  Greeting.java
pradip@ProBook:~/assignment_1_cloud-$ git branch
* main
pradip@ProBook:~/assignment_1_cloud-$ git checkout -b branch_b
Switched to a new branch 'branch_b'
pradip@ProBook:~/assignment_1_cloud-$ git branch
* branch_b
  main
pradip@ProBook:~/assignment_1_cloud-$ ls
calculator.py  Greeting.java
pradip@ProBook:~/assignment_1_cloud-$ nano calculator.py
pradip@ProBook:~/assignment_1_cloud-$ nano Greeting.java
```

3. Commit the Changes: B stages and commits their progress locally:

```
pradip@ProBook:~/assignment_1_cloud-$ git add *
pradip@ProBook:~/assignment_1_cloud-$ git commit -m "Commit from developer B"
[branch_b 30859f5] Commit from developer B
 2 files changed, 2 insertions(+)
```

4. Push the Branch to GitHub: B uploads the new branch to GitHub repository:

Creating the Pull Request (On GitHub Website)

```
pradip@ProBook:~/assignment_1_cloud-$ git push -u origin branch_b
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 438 bytes | 438.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'branch_b' on GitHub by visiting:
remote:      https://github.com/pradipkhendake/assignment_1_cloud-/pull/new/branch_b
remote:
To https://github.com/pradipkhendake/assignment_1_cloud-.git
 * [new branch]      branch_b -> branch_b
branch 'branch_b' set up to track 'origin/branch_b'.
```

Once the branch is pushed, the rest of the work happens in the web browser.

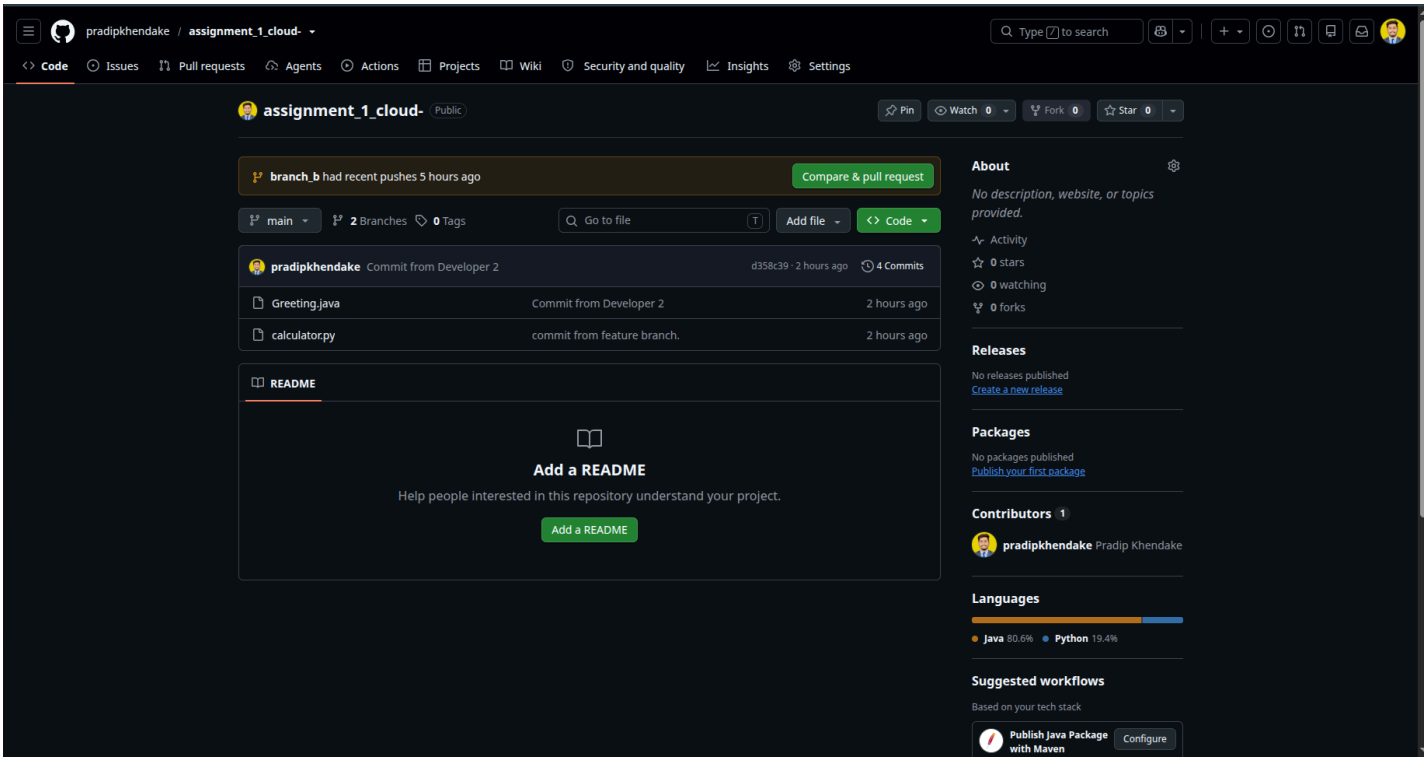
1. Navigate to GitHub: Open the repository page on GitHub.

2. Click the Banner: A yellow banner will automatically appear at the top saying: "Tracked branch recently pushed...". Click the green Compare & pull request button.

3. Fill Out the Details:

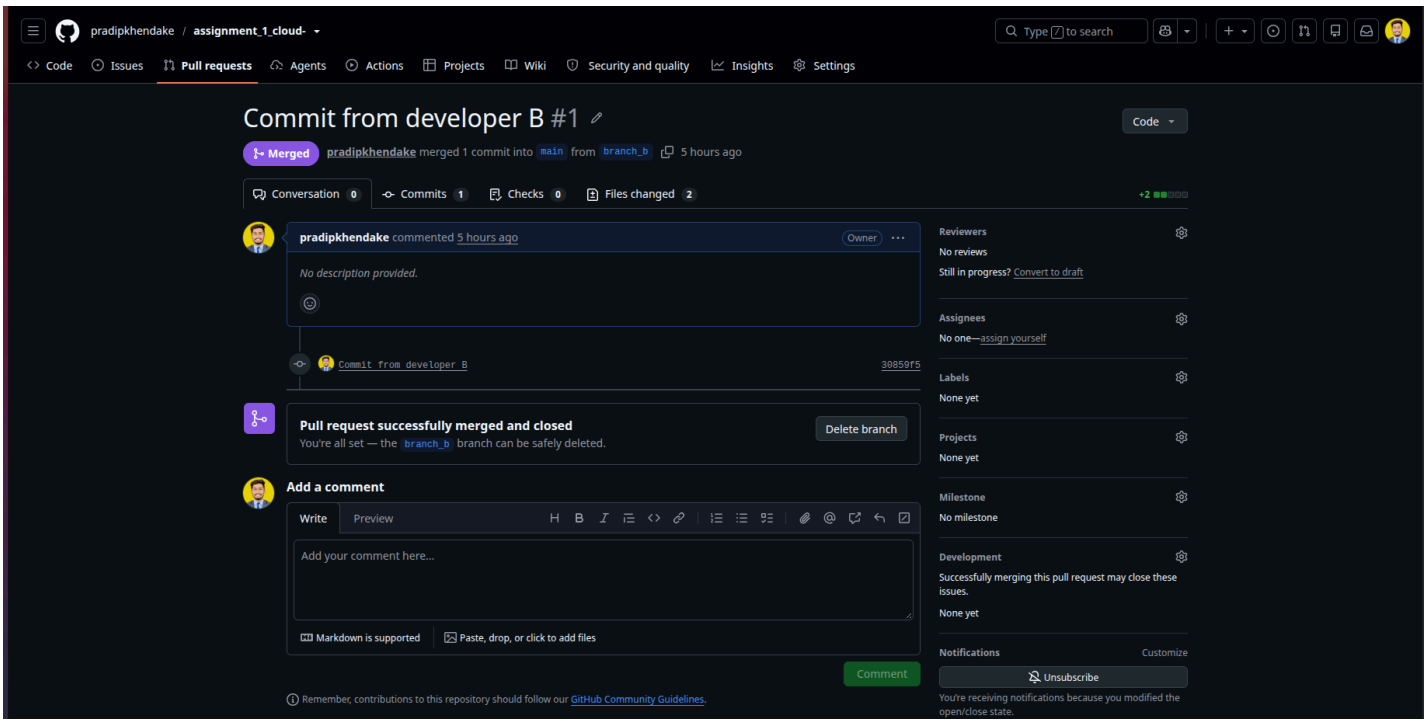
o Title: Write a short summary of the change (e.g., "Add user login feature").

o Description: Briefly explain what files were changed and how to test the code.



4. Submit: Click the green Create pull request button.

Code Review & Merging (For A)
A (the repository owner) now takes over.
1. Review the Code: A goes to the Pull Requests tab, clicks on the open PR, and clicks Files changed to review the lines of Java or Python code.
2. Merge the PR: If the code looks correct and tests pass, A clicks the green Merge pull request button, then clicks Confirm merge.



3. Clean Up: Click Delete branch on GitHub to keep the repository tidy.

```
pradip@ProBook:~/assignment_1_cloud-$ git branch
* branch_b
  main
pradip@ProBook:~/assignment_1_cloud-$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
pradip@ProBook:~/assignment_1_cloud-$ git branch -d branch_b
warning: deleting branch 'branch_b' that has been merged to
         'refs/remotes/origin/branch_b', but not yet merged to HEAD
Deleted branch branch_b (was 30859f5).
pradip@ProBook:~/assignment_1_cloud-$ git branch
* main
```

Syncing local repository

Now that the repository has changed, both A & B need to update their local machines.

Switch back to the main branch locally

git checkout main

Download the newly merged changes from GitHub

git pullrecent:

```
pradip@ProBook:~/assignment_1_cloud-$ git branch
* main
pradip@ProBook:~/assignment_1_cloud-$ ls
calculator.py  Greeting.java
pradip@ProBook:~/assignment_1_cloud-$ cat calculator.py
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        print(a - b)
        return

add(a,b)
pradip@ProBook:~/assignment_1_cloud-$ cat Greeting.java
import java.util.Scanner;

public class Greeting {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your username: ");
        String username = scanner.nextLine();

        System.out.println("Hey there, Friend!");

        scanner.close();
    }
}
```

```
pradip@ProBook:~/assignment_1_cloud-$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 909 bytes | 909.00 KiB/s, done.
From https://github.com/pradipkhendake/assignment_1_cloud-
 * branch            main       -> FETCH_HEAD
   d358c39..af47881  main       -> origin/main
Updating d358c39..af47881
Fast-forward
 Greeting.java | 1 +
 calculator.py | 1 +
 2 files changed, 2 insertions(+)
pradip@ProBook:~/assignment_1_cloud-$ cat calculator.py
#!/bin/python3
a=3
b=5

def add(a, b):
        print(a + b)
        print(a - b)
        print(a * b)
        return

add(a,b)
pradip@ProBook:~/assignment_1_cloud-$ cat Greeting.java
import java.util.Scanner;

public class Greeting {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your username: ");
        System.out.print("Welcome friend !!");
        String username = scanner.nextLine();

        System.out.println("Hey there, Friend!");

        scanner.close();
    }
}
```